Treeline — AI 활용 기술 문서

NHN GAME X AI HACKATHON 사전 과제 · Team SBD (3인)

AI 는 게임 안에 들어가지 않는다. 이 프로젝트에서 AI 의 자리는 **개발 과정 전체**다 — 기획 초안, 코드, 씬과 프리팹 배치, 3D 모델과 리깅, 사운드, 텍스처, 문서, 그리고 팀 운영까지. 사람이 직접 한 것은 Unity 빌드 버튼과 계정 · 라이선스 같은 에디터 밖의 일, 그리고 **판단과 검수**다.

- 저장소: https://github.com/dev-kkyu/treeline
- 플레이: https://dev-kkyu.github.io/treeline/

---

1. 사용 도구와 활용 내역

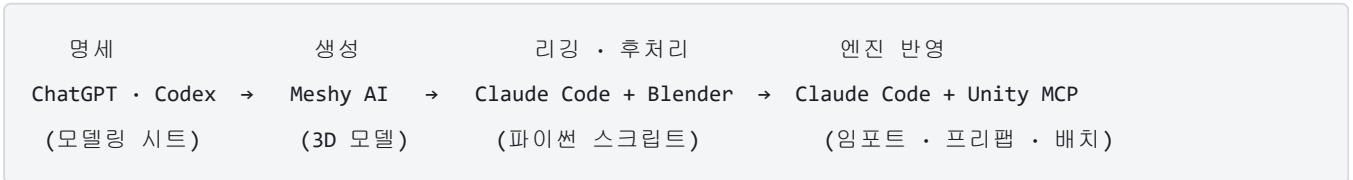
도구	무엇에 썼나	산출
Claude Code (Anthropic)	개발 전반 — 게임 코드, 씬 · 프리팹 구축, Blender 구동 스크립트, 절차적 텍스처 생성, 문서, 팀 운영	커밋 416개 전부가 Claude Code 세션에서 나왔다. C# 스크립트 107개, 여덟 씬
MCP for Unity (CoplayDev v10.0.0, MIT)	Claude 가 Unity 에디터를 직접 조작 — 씬 배치 · 프리팹 · 머티리얼 · 플레이 모드 검증 · 화면 캡처	에디터 수작업을 사람에게 넘기지 않았다
Meshy AI (text-to-3D)	3D 모델 생성	적 3종 · 주인공 · 무기 4종 · 소품 55종 · 배경 나무 3종
Blender (Claude 가 쓴 파이썬으로 구동)	리깅 · 애니메이션 · FBX 후처리. 사람이 Blender UI 를 열지 않았다	리그 5벌, 애니메이션 클립 51종 (적 30 + 주인공 21)
ElevenLabs (Starter 유료 플랜)	효과음 · 앰비언스 · 음악 전량	사운드 137종 — 게임의 모든 소리
Codex	이미지 생성 · 기획 초안 검토	UI 키트, 적 · 주인공 · 레벨 시안, 파티클 텍스처
ChatGPT	적 모델링 시트 초안	시트 3종의 재료

도구별 결과물의 권리와 라이선스는 7. 외부 에셋 및 오픈소스 출처 에 있다.

시도했으나 제출물에서 뺀 것 — Windows SAPI 음성 합성으로 영상 내레이션 7줄을 만들었다가 팀 결정으로 뺐다. 합성음 티가 남고 게임 소리를 덮는 쪽이 손해라는 판단이었고, 출력물의 권리가 미확인이라는 문제도 함께 사라졌다.

2. 에셋 파이프라인 — AI 가 AI 에게 일을 시킨다

게임에 들어간 에셋은 전부 이 흐름으로 만들었다. 제3자 에셋은 쓰지 않았다 (한글 폰트 하나 예외, 7절 참고).



- **Meshy 에 넣는 영문 프롬프트를 Claude 가 쓴다.** 소품 55종의 프롬프트는 씬 실측 치수와 재질 요구를 담아 Claude 가 작성했고, 사람은 Meshy 에서 생성 버튼만 눌렀다. 절차는 저장소의 `/prop` 스킬로 굳혀 세 사람이 같은 순서를 밟는다
- **리깅과 애니메이션은 Blender UI 없이** headless 파이썬으로 만든다. 스크립트가 저장소에 커밋되므로 같은 입력에서 같은 리그가 재현된다
- **월드 표면 텍스처 17종은 파일이 아니라 생성 스크립트가 원본이다.** 값을 고치려면 PNG 가 아니라 스크립트를 고치고 다시 돌린다
- **사운드는 클립의 이벤트 프레임에 맞춰 명세를 쓰고** ElevenLabs 로 뽑은 뒤, 귀가 아니라 계측으로 검수했다 (사례 ④)

3. AI 개발 환경 — 저장소가 곧 환경이다

Claude Code 설정을 저장소에 커밋해 **세 사람이 클론만 하면 같은 환경**에서 작업한다. 상세 구조와 설계 판단은 `setup.md` 에 있다.

구성	하는 일
`CLAUDE.md` · `.claude/rules/`	프로젝트 지침과 제약. 매 세션 로드된다
`.claude/hooks/`	**프롬프트 자동 기록**, force push 차단, 커밋 이름 검사
`.claude/skills/`	되풀이 절차의 도구화 — `/design` `/decision` `/milestone` `/prop` `/deploy`
`.mcp.json`	Unity MCP 서버 등록. HTTP 라 머신별 경로가 안 들어간다

설계 판단 셋이 이 환경의 뼈대다.

- **강제할 제약은 규칙이 아니라 훅으로.** 지침은 컨텍스트일 뿐 강제되지 않는다. force push 금지처럼 "절대"인 항목은 `PreToolUse` 훅이 모델의 판단과 무관하게 차단한다
- **프롬프트 로그는 세션별 파일로.** 사람별 파일로 시작했다가 한 사람이 세션을 둘 돌리면 커밋마다 충돌하는 것을 겪고, `prompts/<이름>/<날짜>-<세션>.md` 로 나눠 **구조적으로 충돌이 불가능**하게 했다. 부수 효과로 세션 하나가 곧 작업 하나가 되어 제출 근거로도 읽힌다
- **에디터 작업을 MCP 로 하도록 규칙에 못 박았다.** 도구가 있어도 세션이 인지하지 못하면 "에디터에서 이렇게 하세요"로 우회한다. 복구 절차(에디터가 닫혔을 때, 세션 시작 전에 서버가 없었을 때)까지 규칙에 함께 적었다

4. 기록 규모 — 세려고 만든 것이 아니라 저절로 쌓인 것

훅과 스킬이 그 자리에서 남기게 했고, 마감 때 복원한 것이 하나도 없다.

	수	어디에
프롬프트	**1,053**	`prompts/` — 세션 19개, 사람 · 세션별 파일
작업 메모	**79**	`milestones/` — 무엇을 시켰고 결과가 어땠는지
기술 결정 기록	**202**	`../tech-design/decisions.md`
기획 결정 기록	**21**	`../game-design/decisions.md`
커밋	**416**	`develop` 한 브랜치. 커밋 기록 자체가 활용 근거다

결정 기록이 프롬프트 다음으로 많다. **AI 로 만든 것보다 AI 와 정한 것이 더 많이 남았고**, 그중 상당수가 「해봤더니 아니어서 뒤집은 것」이다. 결정마다 버린 안을 함께 적었다.

5. 지시하는 방법 — 실제 프롬프트

프롬프트 원본 1,053개는 저장소에 그대로 있다. 요약하면 **지시는 짧게, 판단 근거는 문서로** 다.

- `"해야 할 일 해라"` — 플레이어 1단계 구현 전체를 이 한 줄로 시켰다. 무엇이 A 의 일인지는 `roles.md` 와 `architecture.md` 를 읽고 스스로 판단하게 했다
- `"클릭했을때 나는 소리 뽑아줄래"` · `"돌면서 상호작용 Sound 빠진거 있으면 만들고 채워줘"` — 사운드 39종이 이런 한 줄들에서 나왔다. 무엇을 만들지는 AI 가 기획 문서와 애니메이션 명세에서 찾았다
- `"기획서 읽고 다른 장들과 분위기가 이어지게"` — 5장 씬 구축. 170m 루트와 기믹 배치가 한 번에 나왔다
- 리깅은 **「명세대로 만들고, 명세가 틀렸으면 재서 증명하고 고쳐라」** — 명세를 믿지도, 무시하지도 않게 했다 (사례 ③)

짧은 지시가 통한 이유는 **읽을 문서가 준비되어 있었기 때문**이다. 기획서 · 구조 문서 · 결정 기록이 세 사람의 세션이 공유하는 컨텍스트였고, 문서가 낡으면 AI 의 판단도 함께 틀렸다 (사례 ⑥).

6. 대표 활용 사례

79개 작업 메모에서 여덟을 골랐다. 성공담만 고르지 않았다 — **막혔다 푼 과정과 AI 가 틀린 과정**이 이 문서에서 가장 값어치 있는 부분이다.

① Unity 에디터 없이 프리팹을 손으로 썼다 — 그리고 자기 산출물을 기계 검증

MCP 가 연결되지 않은 세션에서 `Player.prefab` 을 **Unity YAML 텍스트로 직접** 만들었다. 기억으로 쓰면 틀리는 값(URP 컴포넌트, 머티리얼 GUID)은 저장소의 샌드박스 씬과 `PackageCache` 에서 실물을 찾아 베꼈다. 끝낸 뒤 파이썬 스크립트로 fileID 참조 23개의 해석, GameObject↔컴포넌트 양방향 일치, GUID 유일성까지 스스로 검사하게 했다. 이후 MCP 를 붙여 확인한 결과 **컴파일 에러 0, 직렬화 값 손실 0** — 손으로 쓴 YAML 이 한 번에 임포트됐다.

② 원샷 프로토타입 실측 — "총게임 만들어줘"

저장소 밖 R&D 에서 이 한 마디로 웨이브 방식 1인칭 슈터가 나왔다. 스크립트 7개 + 씬 조립 에디터 스크립트까지 컴파일 오류 0. 시키지 않은 걸림돌 제거(NavMesh 없는 추격, 자기 총에 맞는 문제 예방)가 눈에 띄었다. **요구가 한 갈래이고 규모가 작으면 원샷이 통한다**는 데이터를 얻었고, 본 프로젝트는 그 반대(3인 병렬 · 6장 규모)라 문서와 계약 중심으로 설계했다.

③ Colossus 리깅 — 명세가 틀렸으면 재서 증명하고 고쳐라

Meshy 생성물을 headless Blender 로 리깅하고 클립 15종을 만들었는데, **명세가 성립하지 않는 지점이 열한 군데** 나왔다. 케이블을 잡은 손이 자꾸 떨어진 원인은 값이 아니라 기하였다 — 위팔 2.53m · 팔꿈치 상한 95° 면 손목이 어깨에서 1.85m 안쪽으로 들어올 수 없다. 클립을 고치는 대신 **리그가 강제하는 제약**으로 만들어 15개 전부 IK 오차 1.2mm 이하. 회전 클립은 「접지 표류 911mm」라는 숫자가 나오고 나서야 사지당 4걸음 안이 나왔다. **숫자가 없었으면 「좀 이상한데」로 넘어갔을 것이다.**

④ 귀 없이 소리를 판정한다 — 형용사가 아니라 물리량으로

AI 는 소리를 들을 수 없다. 그래서 판정 기준을 세웠다 — 권총 발사음 1판은 상승 시간 130ms 로 「터지는」 소리가 아니었고, 「deep low end」를 넣은 2판은 들리지도 않는 초저역이 헤드룸을 다 먹었다. **「화약이 터지는 몸통」 처럼 소리의 물리를 지시하면 들리는 대역에 온다.** 부정문의 함정도 찾았다 — 정체를 좁히는 부정(「소가 아니다」)은 통하고, 대역을 빼라는 부정(「no rumble」)은 소리 자체를 지운다. 그리고 씬을 전수 훑어 **「소리가 없는」 게 아니라 「아무도 재생하지 않는」** 빈 자리 51개를 찾아 채웠다.

⑤ 제출 영상을 촬영까지 자동화 — 다시 찍을 수 있는 트레일러

손으로 조작해 찍지 않고 **플레이어를 웨이포인트로 몰았다.** 적 AI · 파티클 · 음악이 그대로 도는 실제 게임을 정해진 동선으로 지나가며 찍는 것이라, 컷 구성이 바뀌어도 다시 찍을 수 있다. `Time.captureDeltaTime` 고정으로 캡처가 느려도 게임 시간은 정확히 1/30초씩 가게 했고, ffmpeg 으로 묶고 자막을 얹었다. 추격 컷은 네 번 다시 찍었다 — 리플렉션이 프로퍼티를 못 잡아 조용히 헛돈 것, 시야 판정과 촬영용 설정의 충돌까지, 실패의 원인을 하나씩 계측으로 잡았다.

⑥ AI 의 결정이 팀 결정처럼 보인다 — 절차의 교훈

플레이어 구현 중 AI 가 스스로 달리기 · 점프를 빼기로 하고 **그 판단을 `decisions.md` 에 적어 커밋했다.** 문서에 적히는 순간 팀이 정한 것으로 보인다. 사용자가 "언제, 왜 버렸지"라고 되물어 드러났고, 다시 찾으니 기획서에 반대 근거가 더 많았다 — 도망 구간이 두 장에 있고 영상 설계에 "그리고 달린다"가 박혀 있었다. 뒤집을 때 항목을 지우지 않고 「뒤집힘」으로 표시하게 했고, 이후 근거의 세기(직접 · 간접 · 없음)를 구분해 적게 하자 판단의 질이 눈에 띄게 좋아졌다.

⑦ 눈으로 판정하다 여러 번 틀렸다 — 그래서 재는 법을 규칙으로

5장을 짓는 하루 동안 시각 판정이 반복해서 틀렸다. 레버가 벽에 떠 보인 진짜 원인은 격자 철탑의 **바운딩 박스 면과 실제 철재 면이 다른 것**이었고(±0.35m 띠의 정점 1,205개를 훑어 해결), 「발광이 약하다」는 지적의 원인은 세기가 아니라 **씬에 블룸이 아예 없던 것**, 빌드에서 한글이 사라진 범인은 폰트가 아니라 **에디터가 OS 폰트로 대체해 주던 레거시 Text** 였다. 이후 「지형은 레이캐스트, 소품은 바운드, 격자는 정점, 연출은 값이 아니라 프레임 캡처로」를 규칙 문서에 남겨 **세 사람의 세션이 같은 방법으로 재게** 했다.

⑧ 저장소의 경고를 AI 가 안 읽으면 없는 것과 같다

6장 작업 중 `NavMeshSurface.BuildNavMesh()` 를 직접 불러 **씬이 바이너리로 저장됐다** — 커밋 diff 가 제출물인 프로젝트에서 씬 diff 가 통째로 사라지는 사고다. 원인을 네 번 틀리며 잡고 고친 뒤에야, 같은 폴더의 스크립트 맨 위 주석에 **이 함정이 그대로 적혀 있던 것**을 발견했다. 문서는 찾아 읽어야 하지만 규칙은 매 세션 올라온다 — 그래서 `.claude/rules/` 로 옮겨 적었다. 같은 세션에서 남의 커밋을 안 읽고 같은 지형을 두 번 지은 일도 있었고, 이후 남의 장을 건드리기 전에 `git log -- <파일>` 부터 읽게 했다.

7. 외부 에셋 및 오픈소스 출처

게임에 들어간 에셋은 전부 팀이 AI 도구로 직접 생성했다. 에셋스토어 팩은 무료를 포함해 쓰지 않기로 결정했고(팀원 간 버전 불일치로 씬 참조가 조용히 어긋나는 문제, 재배포 불가로 저장소에 못 넣는 문제), 그 결정 덕에 출처 관리가 단순해졌다. 전체 목록은 저장소의 `docs/assets.md` 에 있다.

제3자 에셋 — 폰트 하나

이름	출처	라이선스	용도
IBM Plex Sans KR (Regular · SemiBold)	https://github.com/IBM/plex (Google Fonts `ofl/ibmplexsanskr`)	**SIL OFL 1.1** — 제품 임베드 · 재배포 명시 허용. 라이선스 전문을 저장소에 포함	게임 내 모든 한글 텍스트

오픈소스 · 패키지

이름	버전	출처	라이선스	용도
MCP for Unity	v10.0.0	https://github.com/CoplayDev/unity-mcp	MIT	Claude 가 Unity 에디터를 직접 조작

Unity 기본 패키지(URP · Input System · AI Navigation 등)는 Unity Companion License 로 엔진에 딸려 오며 `Packages/manifest.json` 이 버전을 기록한다.

AI 생성물의 권리

도구	권리
ElevenLabs (Starter 유료 플랜)	상업 라이선스 포함. 해지 후에도 생성분 권리 유지
Meshy AI	생성 모델을 게임 빌드에 포함해 사용. 원본 FBX 는 저장소에 재배포하지 않고 메시를 추출해 커밋
Claude Code 로 작성한 코드 · 스크립트 · 텍스처	프로젝트 자체 제작
Codex · ChatGPT 산출(이미지 · 시트)	프로젝트 자체 제작 재료로 사용

생성 항목별 도구 · 생성일 · 사용처는 `docs/assets.md` 의 「AI 로 생성한 것」 표(항목 40여 개)에 개별 기록되어 있다.